

The Java Iterator

Traversing Collections with Precision

CODEHIVE

CODEHIVE

1. What is an Iterator?

An **Iterator** is an object that allows you to traverse a collection, one element at a time. While "looping" is the general term, "iterating" specifically refers to using the Iterator interface to navigate data structures like `ArrayList`, `HashSet`, and more.

Why use an Iterator over a For-Loop?

While a `for-each` loop is cleaner, an Iterator provides more control, specifically the ability to modify the collection (remove items) while the loop is still running.

2. Getting an Iterator

Every Collection in Java provides an `iterator()` method that returns an instance of the Iterator interface pre-loaded with your data.

```
import java.util.ArrayList;
import java.util.Iterator;

ArrayList<String> cars = new ArrayList<>();
cars.add("Volvo");
cars.add("BMW");
cars.add("Ford");

// Get the iterator
Iterator<String> it = cars.iterator();

// Grab the first item
System.out.println(it.next()); // Output: Volvo
```

3. The Traversal Pattern

To go through an entire collection, we use a while loop combined with two critical methods: `hasNext()` and `next()`.

1. **`.hasNext()`**: Returns `true` if there is at least one more element to see.
2. **`.next()`**: Returns the next element and moves the "pointer" forward.

```
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

4. Safe Deletion

One of the most powerful features of the Iterator is the `remove()` method. This is the **only safe way** to remove an item from a collection while you are looping through it.

```
Iterator<Integer> it = numbers.iterator();
while(it.hasNext()) {
    Integer i = it.next();
    if(i < 10) {
        it.remove(); // Safely deletes from 'numbers'
    }
}
```

5. The Concurrent Modification Pitfall

A common mistake in Java is trying to remove an item using `list.remove()` inside a standard for-each loop.

DANGER:

If you use `for(String s : list) { list.remove(s); }`, Java will throw a `ConcurrentModificationException`. This happens because the loop's internal counter gets out of sync with the list's actual size.

Always use an Iterator for deletions.

6. Modern Iterator Syntax

Following Java 10 standards, you can use the `var` keyword to make your Iterator declarations less verbose, especially when dealing with complex generic types.

```
// Traditional way
Iterator<String> it = cars.iterator();

// Modern way (Java 10+)
var it = cars.iterator();
```

7. Iterator Life Cycle

An Iterator is a "one-time use" object. Once you have called `next ( )` until `hasNext ( )` is false, you cannot "reset" it.

To traverse the collection a second time, you must request a **new** iterator by calling `collection.iterator ( )` again.

8. Iterator vs ListIterator

While `Iterator` works on all collections, `ArrayList` and `LinkedList` also support a `ListIterator`.

- **Iterator:** Forward direction only. Works on Sets and Lists.
- **ListIterator:** Bi-directional (can go `previous()`). Can also `add()` items during iteration. Only works on Lists.

9. Summary & Best Practices

1. Use **Iterators** when you need to remove items during a loop.
2. Use **for-each** loops when you only need to read data (it's cleaner).
3. Remember that `next ( )` actually moves the cursor—don't call it twice in one loop unless you intend to skip elements!

© 2026 CodeHive Academy | The Collection Framework Series